

```

    }
}

```

`main()` will instantiate and `start()` all these threads. After they have started, the methods immediately return and the thread `main()` dies. However the entire application doesn't die until the last thread does. By default, a new thread has the same priority as the thread that created it.

Notice what happened here—in our `main` method, we instantiated our `MyThread` class. Then, we executed the `start()` method of our object. We never called the `run()` method but, lo and behold, it got called.

So, how many threads did we give life to in that example?

```

4
main
chuck,
dede,
pat

```

So, using threads is much like running a road race:

Before running, you must start

1.Understanding the sequence:

My program is running along happily. Without me doing anything, my program is already running in a thread—`main`. (Therefore, your programs have always been running in a single thread.)

2.In this context—where a single thread (`main`) is already running—you're going to execute the following code:

```

public static void main( String[] args )
{
    MyThread mt = new MyThread();
    mt.start();
}

```

Firstly, understand that your goal is to start up a new thread. When you instantiate a new thread object, and then execute its `start()` method, you are asking the JVM to create a new, additional thread to run alongside the thread you created when your original `main()` method started up.

We say that `start()` launches a new thread, and then returns to the calling program, thereby permitting the calling program to continue working independently of the just-launched thread.

Although you have a method `run()` in your `MyThread` class, don't ever execute the `run()` method yourself. Let the JVM do it for you after you execute